

FESTIVAL UNIVERSITARIO DE MÚSICA – ANÁLISIS

Laboratorio 2 - Arreglos y excepciones

Requisitos Funcionales del Sistema:

1. El sistema debe solicitar al usuario, al iniciar el programa, la información del festival: nombre, código de identificación y nombre del coordinador.
2. El sistema debe permitir crear un nuevo festival que reemplace la instancia actual, inicializándolo sin escenarios configurados y sin artistas registrados.
3. El sistema debe almacenar los escenarios del festival utilizando un arreglo básico de objetos, con un tamaño fijo máximo de 5 posiciones.
4. El sistema debe almacenar, para cada escenario, código, nombre, ubicación, capacidad máxima de asistentes y estado.
5. El sistema debe permitir configurar un nuevo escenario en una posición del arreglo indicada por el usuario, validando previamente que dicha posición esté dentro de los límites del arreglo, que se encuentre disponible (que contenga null) y que la información ingresada sea válida.
6. El sistema debe validar que la capacidad máxima de asistentes de un escenario sea mayor que 0, tanto al crearlo como al modificarlo, generando una `IllegalArgumentException` en caso contrario.
7. El sistema debe reconocer que una posición sin escenario configurado contiene null, y debe verificar esta condición antes de invocar cualquier método sobre dicha posición.
8. El sistema debe permitir consultar (listar) todos los escenarios configurados, excluyendo de la consulta las posiciones que contengan null.
9. El sistema debe permitir consultar un escenario específico indicando su posición, informando al usuario si la posición no es válida o se encuentra vacía. El programa no debe detenerse.

10. El sistema debe permitir modificar la capacidad máxima y el estado de un escenario previamente configurado, impidiendo la modificación de posiciones con null y validando que la nueva capacidad sea mayor que 0.
11. El sistema debe permitir retirar (eliminar) un escenario previamente configurado, restableciendo la posición correspondiente del arreglo a null.
12. El sistema debe almacenar los artistas del festival utilizando un ArrayList (no debe usarse la interfaz List), permitiendo registrar una cantidad indeterminada de artistas durante la ejecución del programa.
13. El sistema debe almacenar, para cada artista, código, nombre artístico, género musical, duración de la presentación (en minutos) y cantidad estimada de asistentes.
14. El sistema debe validar que la duración de la presentación sea mayor que 0 y que la cantidad estimada de asistentes no sea negativa, tanto al crear como al modificar un artista, generando una `IllegalArgumentException` en caso contrario.
15. El sistema debe permitir registrar un nuevo artista en el ArrayList, impidiendo el registro de dos artistas con el mismo código.
16. El sistema debe permitir consultar (listar) todos los artistas registrados, gestionando adecuadamente el caso en que el ArrayList esté vacío.
17. El sistema debe permitir buscar un artista mediante su código, recorriendo el ArrayList de forma secuencial.
18. El sistema debe permitir modificar la información de un artista previamente registrado, aplicando las mismas validaciones utilizadas en su creación.
19. El sistema debe permitir cancelar la participación de un artista, localizándolo en el ArrayList mediante su código y eliminándolo de dicha estructura.
20. El sistema debe calcular la cantidad de escenarios configurados y la cantidad de espacios disponibles en el arreglo.
21. El sistema debe determinar el escenario con mayor capacidad máxima de asistentes.

22. El sistema debe calcular la cantidad de artistas registrados, determinar el artista con la presentación de mayor duración, el artista con mayor cantidad estimada de asistentes, y calcular el promedio de duración de las presentaciones.
23. El sistema debe considerar, para los cálculos relacionados con artistas, únicamente los objetos existentes en el ArrayList, evitando ejecutar dichos cálculos cuando este se encuentre vacío.
24. El sistema debe presentar un reporte del festival que muestre de forma conjunta los resultados de los cálculos anteriores (escenarios y artistas).
25. El sistema debe manejar la excepción InputMismatchException cuando el usuario ingrese, mediante Scanner, un valor numérico con un tipo incorrecto, limpiando la entrada inválida y permitiendo la continuidad del uso del programa.
26. El sistema debe utilizar bloques try-catch para manejar las excepciones producidas durante la interacción con el usuario y durante la validación de los objetos (escenarios y artistas).
27. El sistema debe utilizar al menos un bloque finally en una operación donde tenga sentido ejecutar una acción final, independientemente de si esta se completa con éxito o produce una excepción.
28. El sistema debe continuar funcionando después de gestionar una entrada incorrecta o una excepción, sin finalizar de forma inesperada.
29. El sistema debe mostrar un menú principal con las siguientes opciones: nuevo festival, configurar escenario, consultar escenarios, consultar un escenario, modificar escenario, retirar escenario, registrar artista, consultar artistas, buscar artista, modificar artista, cancelar participación, mostrar reporte del festival y salir (al salir se finalizará el programa)

Propósito de cada una de las clases del programa:

	Clase	Tipo	Propósito
1	Escenario	Modelo	Representa un espacio físico del festival. Encapsula los datos propios de un escenario (código, nombre, ubicación, capacidad máxima de asistentes y estado) y los expone mediante getters y setters. Es el tipo de objeto almacenado en el arreglo básico administrado por Festival.
2	Artista	Modelo	Representa a un artista participante del festival. Encapsula los datos propios de su presentación (código, nombre artístico, género musical, duración en minutos y cantidad estimada de asistentes) y los expone mediante getters y setters. Es el tipo de objeto almacenado en el ArrayList administrado por Festival.
3	Festival	Modelo	Representa la organización general del evento. Almacena el nombre, código de identificación y coordinador del festival, y administra directamente el arreglo básico de Escenario y el ArrayList de Artista. Provee métodos para agregar, obtener, reemplazar y eliminar elementos de ambas estructuras, delegando la validación de reglas de negocio a los controladores correspondientes.
4	ControladorEscenario	Controlador	Concentra la lógica de validación de los datos de un escenario, verificando que la capacidad máxima de asistentes sea mayor que 0 al crearlo o modificarlo, y generando IllegalArgumentException cuando no se cumple esta regla.
5	ControladorArtista	Controlador	Concentra la lógica de validación de los datos de un artista, verificando que la duración de la presentación sea mayor que 0 y que la cantidad estimada de asistentes no sea negativa, tanto al registrarlo como al modificarlo, generando IllegalArgumentException cuando no se cumplen estas reglas.
6	ControladorFestival	Controlador	Es el controlador principal del sistema y el encargado de orquestar el CRUD completo de escenarios y artistas. Contiene la instancia de Festival, delega las validaciones específicas a ControladorEscenario y ControladorArtista, y coordina la comunicación con VistaFestival para obtener datos del usuario y mostrar resultados. Implementa el ciclo del menú principal, ejecuta cada opción del sistema, realiza los recorridos y cálculos sobre el arreglo y el ArrayList, y maneja las excepciones del programa mediante bloques try-catch y finally.
7	VistaFestival	Vista	Gestiona la entrada y salida de información con el usuario mediante Scanner. Ofrece métodos genéricos de lectura (texto, enteros, decimales, opción de menú) y de presentación (mensajes, menú de bienvenida, información de escenarios y artistas, listados y reporte del festival).
8	Main	Driver program	Punto de entrada de la aplicación. Su única responsabilidad es crear una instancia de ControladorFestival e invocar su método iniciar().

Valores iniciales de los objetos y sus validaciones

Los valores iniciales se proveen a través de los constructores de cada clase, los cuales reciben como parámetros toda la información obtenida previamente desde VistaFestival. Antes de invocar al constructor, ControladorFestival delega la validación de esos datos a ControladorEscenario o ControladorArtista, según corresponda; solo si la validación es exitosa se procede a crear el objeto y agregarlo a la estructura correspondiente (arreglo o ArrayList).

Los valores que deben validarse son:

- **Escenario:** la capacidad máxima de asistentes debe ser mayor que 0 (tanto al crear como al modificar); adicionalmente, el código, nombre y ubicación no deben estar vacíos.
- **Artista:** la duración de la presentación debe ser mayor que 0, y la cantidad estimada de asistentes no puede ser negativa (tanto al crear como al modificar); adicionalmente, el código, nombre artístico y género musical no deben estar vacíos, y el código no debe repetirse con el de otro artista ya registrado.

Cuando alguna de estas reglas no se cumple, se genera una `IllegalArgumentException`, la cual es capturada por ControladorFestival para informar el error al usuario sin detener la ejecución del programa.

Array con escenario o vacía:

Antes de operar sobre cualquier posición del arreglo escenarios, ControladorFestival obtiene el objeto mediante `festival.getEscenario(posicion)` y compara la referencia obtenida contra null utilizando una estructura condicional (`if (festival.getEscenario(posicion) != null)`).

- Si la posición contiene una referencia distinta de null, se considera que el escenario está configurado y se procede a usar sus métodos con normalidad (mostrarlo, modificarlo o retirarlo).
- Si la posición contiene null, se considera vacía; el programa evita invocar cualquier método sobre ella (lo cual generaría una `NullPointerException`) y en su lugar informa al usuario mediante `VistaFestival.mostrarMensaje()` que la posición no tiene un escenario configurado.

Esta misma validación se aplica al recorrer el arreglo completo (por ejemplo, en `consultarEscenarios()`, `calcularCantidadEscenariosConfigurados()` u

obtenerEscenarioMayorCapacidad()), donde cada posición se revisa dentro del ciclo for antes de acceder a sus atributos, omitiendo las posiciones en null.

Operaciones de búsqueda, modificaciones y operaciones:

Búsqueda: se recorre el ArrayList de artistas mediante un ciclo for-each, comparando el código de cada artista (artista.getCodigo()) contra el código ingresado por el usuario mediante .equals(). Si se encuentra una coincidencia, se guarda la referencia al artista encontrado y se detiene la búsqueda; si el ciclo finaliza sin encontrar coincidencias, se lanza y captura una excepción para informar al usuario que el artista no existe.

Modificación: primero se ejecuta la búsqueda descrita anteriormente para obtener la referencia al artista. Si el artista existe, se solicitan los nuevos valores a la vista y se delega la validación y actualización a ControladorArtista.modificarDatosArtista(), el cual aplica las mismas reglas de negocio usadas en la creación antes de invocar los setters correspondientes sobre el objeto encontrado.

Eliminación: de igual forma, se busca primero el artista por código. Si existe, se invoca festival.eliminarArtista(artista), método que internamente utiliza remove() de ArrayList para quitar el objeto de la colección. Todo este proceso se envuelve en un bloque try-catch-finally, donde el finally confirma que el proceso de cancelación terminó, independientemente de si el artista fue encontrado y eliminado o no.

Situaciones en las que se pueden producir excepciones:

Situación	Excepción	Dónde se maneja (try-catch)
El usuario ingresa un valor no numérico cuando se solicita un entero (posición, capacidad, duración, asistentes, opción de menú)	InputMismatchException	En VistaFestival.leerEntero(), limpiando el buffer del Scanner con next() o nextLine() y solicitando el dato nuevamente
Se intenta crear o modificar un escenario	IllegalArgumentException	En ControladorEscenario (validarDatosEscenario() y modificarDatosEscenario()), capturada en

Universidad del Valle de Guatemala
 CC2008 - Introducción a la Programación Orientada a Objetos
 Semestre II, 2026

con capacidad máxima menor o igual a 0		ControladorFestival.configurarEscenario() y modificarEscenario()
Se intenta registrar o modificar un artista con duración menor o igual a 0, o con asistentes negativos	IllegalArgumentException	En ControladorArtista (validarDatosArtista() y modificarDatosArtista()), capturada en ControladorFestival.registrarArtista() y modificarArtista()
El usuario indica una posición fuera del rango del arreglo de escenarios (menor que 0 o mayor que 4)	ArrayIndexOutOfBoundsException (o validación explícita equivalente)	En ControladorFestival.configurarEscenario(), consultarEscenario(), modificarEscenario() y retirarEscenario()
El usuario indica una posición del arreglo que ya está ocupada al configurar un escenario	Excepción personalizada o validación con IllegalStateException	En ControladorFestival.configurarEscenario()
El usuario intenta consultar, modificar o retirar una posición del arreglo que contiene null	Validación explícita (evita NullPointerException)	En ControladorFestival.consultarEscenario(), modificarEscenario() y retirarEscenario()
El usuario intenta registrar un artista con un código ya existente	Excepción personalizada o IllegalStateException	En ControladorFestival.registrarArtista()
El usuario busca, modifica o cancela un artista con un código que no existe en el ArrayList	Excepción personalizada (por ejemplo NoSuchElementException)	En ControladorFestival.buscarArtistaPorCodigo(), modificarArtista() y cancelarParticipacion()
Se intenta calcular el promedio de duración cuando el ArrayList de artistas está vacío	ArithmeticException (división entre cero)	En ControladorFestival.calcularPromedioDuracion()

Uso de finally: se utilizará al menos en configurarEscenario(), retirarEscenario(), registrarArtista() y cancelarParticipacion(), donde el bloque finally mostrará un mensaje confirmando que el proceso (de configuración, retiro, registro o cancelación) finalizó, independientemente de si la operación se completó con éxito o se produjo una excepción durante su ejecución.